

langGraph_debugging_strategy_and_probleme_faced

For LangGraph applications, debugging is much easier if you debug **one layer at a time**. Your current code has **three layers**:

1. Python code
2. LangGraph routing
3. LLM behaviour

If you try to debug all three simultaneously, it's difficult to identify the source of a problem.

Strategy 1 — Debug the graph state (most important)

At the beginning and end of every node, print the state.

```
from pprint import pprint

def hotel_agent(state: GraphState):

    print("\n===== HOTEL AGENT =====")
    pprint(state)

    ...

    print("\nResponse:")
    print(response)

    return {
        "messages": [response]
    }
```

Do the same for every node.

```
def supervisor(state):

    print("\n===== SUPERVISOR =====")
    pprint(state)

    ...
```

This lets you answer:

- Is messages changing?
- Is next_agent changing?
- Is hotel ever updated?
- Is flight_reserved ever becoming True?

Strategy 2 — Print every message

Often the issue is hidden in the conversation history.

```
for i, msg in enumerate(state["messages"]):
    print(f"\n----- Message {i} -----")
    print(type(msg))
    print(msg)
```

or

```
for msg in state["messages"]:
    msg.pretty_print()
```

Strategy 3 — Print tool calls

Instead of only

```
response = hotel_llm.invoke(messages)
```

print

```
print(response.content)
print(response.tool_calls)
```

Example

Content:

Tool calls:

```
[
  {
    'name': 'search_hotel',
    'args': {'country': 'Spain'}
  }
]
```

If

```
response.tool_calls == []
```

then your LLM never called the tool.

Strategy 4 — Print ToolNode outputs

After the tool executes

Hotel Agent

↓

ToolNode

↓

Hotel Agent

inspect the messages again.

```
for msg in state["messages"]:
    msg.pretty_print()
```

You should see

HumanMessage

AIMessage
(tool call)

ToolMessage
(tool result)

If the ToolMessage is missing, ToolNode isn't executing correctly.

Strategy 5 — Use streaming

Instead of

```
result = app.invoke(initial_state)
```

use

```
for event in app.stream(initial_state):

    print("\n=====")
    print(event)
```

This is one of the best debugging methods because you can see the graph executing step by step.

Example

Supervisor

↓

Hotel Agent

↓

Hotel Tool

↓

Hotel Agent

↓

Supervisor

↓

Flight Agent

instead of waiting until the end.

Strategy 6 — Stream with values

Even better

```
for event in app.stream(
    initial_state,
    stream_mode="values"
):
    print(event)
```

Now you'll see the complete state after every node.

Strategy 7 — Compile with debugging enabled

```
app = graph.compile(debug=True)
```

LangGraph prints information about node execution and transitions.

Strategy 8 — Print routing decisions

Inside

```
def route_supervisor(state):
```

write

```
def route_supervisor(state):

    print("Routing to:", state["next_agent"])

    return state["next_agent"]
```

This verifies that the supervisor's decision matches the graph routing.

Strategy 9 — Print the exact LLM prompt

Before calling

```
hotel_llm.invoke(messages)
```

print the messages:

```
for m in messages:
    print("=" * 40)
    print(type(m))
    print(m.content)
```

Many bugs are caused by the prompt not containing the expected context.

Strategy 10 — Test each node independently

You don't need the graph to debug a node.

```
state = {
    "messages": [
        ("user", "Find me a hotel in Spain")
    ],
    "next_agent": "",
    "hotel": "",
    "flight_reserved": False,
    "vacation_requested": False,
}

print(hotel_agent(state))
```

If this fails, the graph isn't the problem.

Strategy 11 — Test each tool independently

```
print(search_hotel.invoke({"country": "Spain"}))
```

Expected output

```
Hotel Ritz Madrid
Barcelona Princess
Gran Hotel Bali
```

Then

```
print(reserve_airplane.invoke({"destination": "Spain"}))
```

Strategy 12 — Replace the LLM temporarily

If you suspect the LLM is making poor decisions, replace it with a fixed response.

```
def supervisor(state):

    return {
        "next_agent": "hotel"
    }
```

If the graph works, the problem is the LLM rather than the graph.

Strategy 13 — Limit recursion

This prevents infinite loops.

```
result = app.invoke(
    initial_state,
    config={
        "recursion_limit": 20
    }
)
```

Instead of looping forever, LangGraph raises an error after 20 node executions.

Strategy 14 — Inspect the checkpoint (when using memory)

If you later add a checkpointer:

```
memory = InMemorySaver()

app = graph.compile(
    checkpointer=memory
)
```

you can inspect the saved state after each step to see how it evolves.

Strategy 15 — Visualise the graph

Generate a diagram of the graph to ensure the edges are correct.

```
print(app.get_graph().draw_ascii())
```

or, if you have Mermaid support:

```
print(app.get_graph().draw_mermaid())
```

This helps spot incorrect or missing edges.

Strategy 16 — Read Python exceptions carefully

Your current tools contain a Python error:

```
@tool
def search_hotel(country: str):

    return {
        "messages": [response],
        "hotel": hotels,
    }
```

`response` is **not defined** anywhere inside the tool.

The same issue appears in:

```
reserve_airplane()
```

and

```
take_vacation()
```

These functions will raise:

```
NameError: name 'response' is not defined
```

A tool should only return its own result, not reference variables from an agent.

Strategy 17 — Understand where state changes

This is the biggest conceptual point.

Your graph state only changes when a **node** returns an update.

For example:

```
def hotel_agent(state):

    ...

    return {
        "messages": [response]
    }
```

This updates `messages`.

The tool

```
@tool
def search_hotel(...):
```

does **not** directly update `GraphState`. With `ToolNode`, its return value becomes a `ToolMessage` appended to `messages`; it doesn't automatically modify fields like `hotel` or `flight_reserved`.

A practical debugging checklist

Whenever something goes wrong, go through these questions in order:

1. **Does the graph enter the expected node?** (Print node names.)
2. **What is the state before the node runs?** (`pprint(state)`.)
3. **What prompt is sent to the LLM?**

4. What does the LLM return? (content and tool_calls.)
5. Does the tool execute?
6. What does the tool return?
7. What does the state look like after the node finishes?
8. Where does the router send execution next?
9. Does the state match your expectations?
10. Repeat for the next node.

Following this sequence isolates whether a problem comes from the graph wiring, state updates, tool execution, or the LLM's reasoning.

You've actually learned several important LangGraph concepts while building this multi-agent system. Here's a summary of the problems you encountered, why they happened, and how you fixed them.

1. State was never updated

Problem

Your GraphState was:

```
class GraphState(TypedDict):
    messages: Annotated[list, add_messages]
    next_agent: str
    hotel: str
    flight_reserved: bool
    vacation_requested: bool
```

But only `messages` was changing.

For example:

```
return {
    "messages": [response]
}
```

so

```
hotel = ""
flight_reserved = False
vacation_requested = False
```

never changed.

Fix

Return state updates from the node:

```
return {
    "hotel": first_hotel
}
```

or

```
return {
    "flight_reserved": True
}
```

2. Thinking ToolNode updates the state

Problem

You expected

```
@tool
def reserve_airplane(...):
    return {
        "flight_reserved": True
    }
```

to modify

```
state["flight_reserved"]
```

It doesn't.

Why

`ToolNode` converts the tool output into a `ToolMessage` .

So

```
{  
    "flight_reserved": True  
}
```

becomes

```
ToolMessage(  
    content="{ 'flight_reserved': True }"  
)
```

Only graph nodes update `GraphState` .

Fix

Update the state in the agent after receiving the `ToolMessage` .

3. Not understanding ToolMessage

Initially you had

```
HumanMessage
```

Then after the tool:

```
HumanMessage  
AIMessage(tool_call)  
ToolMessage(...)
```

You learned to detect:

```
if isinstance(last_message, ToolMessage):
```

and then update the state.

4. NameError

You wrote

```
ToolMessage
```

without importing it.

Error:

```
NameError:  
ToolMessage is not defined
```

Fix:

```
from langchain_core.messages import ToolMessage
```

5. Wrong order inside the agent

Initially you had

```
return {  
    "hotel": first_hotel  
}  
  
response = hotel_llm.invoke(...)
```

The code after `return` never executes.

You learned that Python exits immediately after `return` .

6. Supervisor always selected hotel

Even after

```
hotel = "Hotel Ritz Madrid"
```

the supervisor kept choosing

```
hotel
```

Why?

Because the supervisor ignored

```
state["hotel"]
```

and only asked the LLM.

7. Supervisor relied only on conversation

Your supervisor did

```
messages = [  
    SUPERVISOR_PROMPT,  
    *state["messages"]  
]
```

The LLM only saw conversation history.

It never saw

```
hotel  
flight_reserved  
vacation_requested
```

So it couldn't reason using the structured state.

8. Wrong Python condition

You wrote

```
elif "" in decision:
```

This is always true because

```
"" in "hotel"
```

returns

```
True
```

Correct version:

```
elif "flight" in decision:
```

or

```
if decision == "flight":
```

9. Infinite loop

Your graph became

```
Supervisor  
↓  
  
Hotel  
  
↓
```



Why?

Because

```
hotel = ""
```

became

```
hotel = "Hotel Ritz Madrid"
```

but the supervisor ignored it.

So it kept routing to the hotel agent.

10. Learning what messages actually stores

Initially you thought

```
messages
```

only contained chat history.

You discovered it actually stores

```
HumanMessage
AIMessage
ToolMessage
AIMessage
ToolMessage
...
```

This is how the LLM knows what happened.

11. Difference between state and messages

One of the biggest lessons was understanding these two concepts.

messages

Contains the conversation.

Example:

```
Human:
Book a hotel.

AI:
```

Calling `search_hotel()`

Tool:
Hotel Ritz Madrid

AI:
Hotel booked.

state

Contains structured data.

Example:

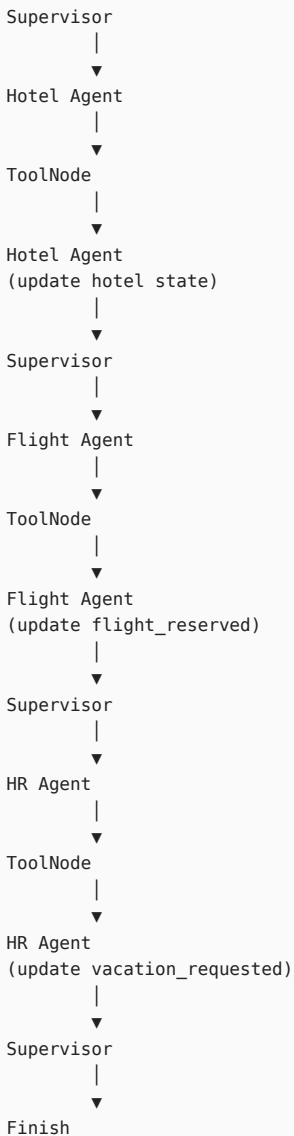
```
{
  "hotel": "Hotel Ritz Madrid",
  "flight_reserved": True,
  "vacation_requested": False
}
```

The LLM reads `messages`.

Your Python code should read and update `state`.

12. Recommended architecture

You ended up with the pattern:



The biggest takeaway

The key concept that ties all of these issues together is:

- **ToolNode** executes tools and appends `ToolMessage`s to `messages`.
- **Only graph nodes** (your agents or dedicated update nodes) can modify `GraphState`.

- The supervisor should make routing decisions based on the structured state (hotel , flight_reserved , vacation_requested) or be explicitly given that state in its prompt if it's LLM-driven.

Once those ideas clicked, the rest of the debugging became much more straightforward.